EMBODIED AGENT PRESENTATION

CHAT-FIRST / ENVIRONMENT-GROUNDED / SKILL-EXTENSIBLE

A Chat-First Embodied Agent Kernel

这不是一个“会聊天的大模型”而已，而是一套能够读环境、承接任务、持续扩展能力的 embodied agent 技术方案。

DESIGN GOAL

让机器人式智能体以对话为入口，但始终扎根于真实环境，而不是停留在纯文本世界。

CORE INSIGHT

真正可落地的 embodied agent，不是更大的框架，而是更清晰的边界与更稳定的系统骨架。

CURRENT EXAMPLES

以 tracking 为主力能力，以 speech 为扩展能力，展示 kernel 如何长出新的 skills。

> “让 agent 在需要的时候读世界、做决定、调能力，而不是把整个系统都塞进一个不可控的大脑里。”

> 这份材料面向非技术受众。它关注的不是函数、脚本和参数，而是这套系统为什么这样设计、它如何协同工作，以及它将如何继续演进。

从“会回答问题”到“能在世界里工作”

传统聊天 agent 擅长理解语言，却并不真正知道周围发生了什么。它能回答“应该做什么”，却很难承担“现在环境里到底发生了什么、下一步该做什么”的连续任务。

THE PROBLEM

当 agent 进入机器人、摄像头、传感器、语音交互这样的真实场景后，仅有语言理解已经不够。它必须能在每一轮决策时看到环境、理解任务是否还在继续、并把新的判断沉淀成下一轮可用的上下文。

THE DESIGN RESPONSE

因此，这个项目并没有把所有复杂性都堆进一个“万能大模型”里，而是刻意做出一个更克制的内核：让环境感知、执行智能体、共享状态和可扩展技能各自承担清晰职责，再围绕一次次 turn 协同工作。

WHAT THE USER FEELS

- 用户说一句话，系统不是只生成回答，而是先理解“现在外部世界是什么状态”。
- 任务不是一轮结束，而是可以继续推进，例如持续跟踪、继续观察、继续播报。
- 新能力不是重新造一套系统，而是在一个稳定内核上长出新的 skills。
- 系统应该是可靠的、可恢复的、可持续扩展的，而不是一次性 demo。

四个主角，一条连续性的骨架

Perception

持续接收相机、雷达、IMU、音频等环境信号，并形成短时间内可读取的世界快照。

↓

Pi Agent

在每一轮中读取上下文，利用 coding agent 基建做判断、选 skill、调工具、产出结果。

↓

Skills

承载 tracking、speech 以及未来更多任务能力，让系统在稳定内核之上持续生长。

↓

Action / Interface

把结果变成机器人动作、语音反馈、可视化界面或面向用户的自然语言回应。

SHARED STATE BACKBONE

一条横向贯穿的共享状态骨架保证系统连续性。它不是另一个“业务模块”，而是让感知、Pi agent、skills 和动作层围绕同一份上下文协同工作的底座。

CHAT-FIRST

对话是触发 turn 的入口。感知不是要取代对话，而是在 turn 发生时提供 grounding。

ONE RUNNER PATH

系统强调一条清晰的决策路径，而不是层层嵌套的 orchestration 框架。

ONE TRUTH SOURCE

长期可共享的状态由系统明确维护，不把系统真相散落在多个隐式缓存里。

Perception 是系统的“世界窗口”

感知层的价值，不在于自己做最终决策，而在于让整个系统在任何时刻都能读到一份“足够新鲜、足够可信”的世界快照。

- 环境信号首先落入 perception，包括相机画面，也为未来的雷达、IMU、音频等多源输入预留统一入口。
- 这些输入不会只是一闪而过，而是被做成短时持久化，从而支持下一轮推理、追踪和回看。
- perception 只负责保留世界状态，不直接承担高层任务编排，这使得它可以保持稳定、持续、独立运行。

WHY THIS MATTERS

WITHOUT PERCEPTION

agent 只能根据语言猜测世界，无法真正知道“目标是不是还在画面里”“环境是否已经变化”“刚才的判断现在是否还成立”。

WITH PERCEPTION

每一轮 turn 都可以站在真实环境之上作出决定。语言、视觉、动作不再是分离的，而是围绕同一份世界快照展开。

Pi Agent 是执行智能体，而不是另一个庞大框架

Pi agent 这一层的核心价值，是充分利用 coding agent 基建，把每一次用户输入变成一轮可执行的 agent turn。

- runner 会在 turn 开始前准备最小但足够的上下文，包括对话、环境摘要、当前共享状态，以及本轮可用的 skills。
- Pi agent 不需要成为“整个系统”，它只需要在这一轮里像一个真正的 coding agent 那样读上下文、理解任务、调用能力、返回结构化结果。
- 这样做的好处是，决策能力可以持续升级，但系统边界依然清楚，外部世界的真相不被藏进一个不可见的大脑里。

ONE TURN STORY

STEP 01

USER INTENT

用户发出一句话，例如“继续跟踪那个人”。

STEP 02

RUNNER FRAMING

系统组装本轮上下文，而不是把 agent 丢进一片空白。

STEP 03

PI READS THE WORLD

Pi 读取对话、环境快照和当前任务状态。

STEP 04

SKILL EXECUTES

Pi 选择最合适的 skill，并调用必要的能力或脚本。

STEP 05

SYSTEM PERSISTS

结果回到共享状态，成为下一轮继续工作的起点。

为什么系统仍然需要一条外部共享状态骨架

即使 Pi agent 或其他 coding agent 具备自己的工作记忆，embodied system 仍然需要外部维护的共享状态。原因不是 agent 不够强，而是系统不能把自己的长期真相藏在单个 agent 的内部上下文里。

IF EVERYTHING LIVES INSIDE PI

状态变成隐式的、不可见的、难恢复的。perception、viewer、robot action 和不同 skills 很难围绕同一份真相协作。

WHEN STATE IS SHARED EXPLICITLY

系统可以连续运行、跨轮恢复、让不同模块共享上下文，并让每一次决策都有清晰的来龙去脉。

WHAT THIS BACKBONE HOLDS

- 当前任务还在不在继续
- 上一轮已经确认了什么目标、做了什么决定
- 最近环境里发生了什么变化
- skills 各自需要延续的状态与结果

ARCHITECTURAL PRINCIPLE

Pi agent 负责思考，但不负责单独保存系统真相。共享状态让系统连续，Pi 让系统聪明，这两者分开，整个 embodied agent 才稳定。

Skills 是能力增长机制，不是插件表演

TRACKING AS THE ANCHOR EXAMPLE

tracking 体现了这套内核最重要的价值：用户用自然语言发起目标，系统在真实环境里寻找、确认、持续跟踪，并把结果回写到连续状态中。它不是一个孤立模型调用，而是一条把对话、环境、技能和动作连成闭环的能力链路。

SPEECH AS THE EXPANSION PROOF

speech 说明这套架构并不只服务于视觉任务。只要 skill 能读到共享上下文、在 Pi agent 的执行空间中运行并输出结构化结果，它就可以自然接入同一个 kernel。

WHAT COMES NEXT

navigation、inspection、问答、告警、任务编排等能力都可以沿着同样的模式扩展，而不必重新设计一套 runtime。

HOW A NEW SKILL FITS IN

- 它不需要自己接管整个系统，只需专注回答“我在这一轮里能做什么”。
- 它读取统一上下文，而不是自己重新收集全部世界信息。
- 它返回清晰结果，再由系统统一写回状态并进入下一轮。

CAPABILITY SURFACE

Tracking | Speech | Navigation | Inspection | Grounded Q&A | Alerting | Task Handoffs | More to Come

从可工作的 kernel，走向完整 embodied platform

Product Roadmap

- 接入更多 skills，让系统从单一示例扩展成真正可组合的能力平台。
- 把 perception 从视觉优先扩展到多传感器融合，让环境理解更稳、更丰富。
- 强化 action 和 interface 层，让系统可以更自然地与机器人设备、语音通道和运营界面打通。
- 增加部署、监控和运行可见性，让它从技术方案走向长期可运营的产品基础设施。

Research Direction

- 更强的长期记忆，让 agent 不只记住一轮任务，而能在更长时间尺度上保持角色与经验。
- 更主动的环境理解，不只是被动读取感知，而是主动寻找证据、主动确认不确定性。
- 更丰富的多模态推理，把语言、视觉、空间和行动放在同一条推理链里。
- 更复杂的人机协作，让系统不仅执行命令，也能在任务推进中给出合理建议和澄清。

CLOSING NOTE

真正的 embodied agent，不是更大的模型，而是更好的系统关系。

当感知持续保留世界，Pi agent 专注每一轮执行，skills 成为能力增长机制，而共享状态提供连续性的骨架，一个可落地的 embodied agent 技术方案才真正成立。